

```

        conn.send('HTTP/1.0 200 OK\r\nContent-type: text/html\r\n\r\n')
        conn.send(response)
        conn.close()

    except OSError as e:
        conn.close()
        print('connection closed')

ap_mode('RPICOWAP',
        'SWEETRPI',
        html,
        led)

```

You can also run your Pico board in a LAN networking mode in case you have set up a 2.4G Wi-Fi network. The code diff given below indicates the changes you need to incorporate in the *picoapobled.py* for running in the Wi-Fi LAN mode, also known as the station mode. Just execute the code after the changes, connect to your Wi-Fi network and make calls like done earlier to turn the onboard LED on/off.

Start of diff for picowsmobled.py

```

18,24c18,26
< def ap_mode(ssid, password, html, led):
<     ap = network.WLAN(network.AP_IF)
<     ap.config(essid=ssid, password=password)
<     ap.active(True)
<
<     while not ap.active():
<         print('Waiting for AP Mode Activation...')
---
> def lan_mode(ssid, password, html, led):
>     wlan = network.WLAN()
>     wlan.active(True)
>     if not wlan.isconnected():
>         print('connecting to network...')
>     wlan.connect(ssid, password)
>
>     while not wlan.isconnected():
>         print('Waiting for connection...')
28a31
>     print('network config:', wlan.ipconfig('addr4'))
30,32c33
<     print('AP Mode Is Active, You can Now Connect')
<     print('IP Address To Connect to:: ' + ap.ifconfig()[0])
<
---
>
72,73c73,74
< ap_mode('RPICOWAP',

```

```

<         'SWEETRPI',
---
>     lan_mode('Wifi Id',
>         'Wifi Password',

```

In fact, you can turn any of the Pico examples given in the last section to get served over a wireless network, with a few changes only.

Let's quickly incorporate the internal temperature readings to be served over the Wi-Fi LAN. A few code lines have been changed in the diff given below to access the temperature reading over the wireless network. Just run the code changes, connect to the network, and access the page (auto refreshing every 5 sec) in your browser by typing the shown IP. Isn't it a cakewalk now for you to use various sensors and control or access their data over the network?

--- Start of diff for picowitemp.py ---

```

8a9
>
11c12
<
<         <head> <title>RPico W</title> </head>
---
>
>         <head> <title>RPico W</title> <meta http-
equiv="refresh" content="5"> </head>
13c14
<
<         <p>%s</p>
---
>
>         <p>Internal Sensor Temperature : %s</p>
18c19,29
< def lan_mode(ssid, password, html, led):
---
> def getitemp():
>     sensor_temp = machine.ADC(4)
>     conversion_factor = 3.3 / (65535)
>     reading = sensor_temp.read_u16() * conversion_factor
>     # The temperature sensor measures the Vbe voltage of a
biased bipolar diode, connected to the fifth ADC channel
>     # Typically, Vbe = 0.706V at 27 degrees C, with a slope
of -1.721mV (0.001721) per degree.
>     temperature = 27 - (reading - 0.706)/0.001721
>     time.sleep(2)
>     return temperature
>
> def lan_mode(ssid, password, html):
45,61d55
<
<         request = str(request)
<         led_on = request.find('/light/on')
<         led_off = request.find('/light/off')
<         print( 'led on = ' + str(led_on))

```